Enterprise Platform Quick Reference

DEVOPS ENGINEERING MASTERY — Volume 16 (Final Volume), Companion Volume. First Edition, 9 November 2026. Bengali-English glossary (150 terms), Kubernetes and GitOps references, Argo CD vs Flux and Helm vs Kustomize comparisons, Backstage and platform design references, multi-cluster and service-mesh decision guides, production checklists, templates and official references — keep this beside your terminal. Kubernetes 1.31+, Argo CD 2.x, Flux 2.x, Helm 3, Backstage 1.x.

হ্যান্ডবুক • শব্দকোষ ১–৬০

Glossary — Advanced DevOps শব্দকোষ (১–৬০)

শব্দ 1–60। প্রতিটি শব্দে ইংরেজি পরিভাষা, বাংলা ব্যাখ্যা ও বাস্তব প্রসঙ্গ দেওয়া আছে।

ক্রম	পরিভাষা (English)	বাংলা ব্যাখ্যা	বাস্তব প্রসঙ্গ / উদাহরণ
1	**Admission Controller**	API server-এ অনুরোধ পৌঁছানোর পর, সংরক্ষণের আগে তা যাচাই বা বদলানো উপাদান।	TechNova-তে OPA Gatekeeper admission controller হিসেবে নীতি-বিরুদ্ধ Pod আটকায়।
2	**Affinity**	Pod-কে নির্দিষ্ট নোড বা অন্য Pod-এর কাছে বা দূরে বসানোর নিয়ম।	ডেটাবেসের replica-গুলো ভিন্ন নোডে বসাতে Pod anti-affinity ব্যবহার করা হয়।
3	**API Aggregation Layer**	তৃতীয় পক্ষের API সার্ভারকে মূল Kubernetes API-তে যুক্ত করার ব্যবস্থা।	Metrics Server এই স্তর দিয়ে metrics.k8s.io API দেয়।
4	**API Server**	Kubernetes control plane-এর প্রবেশদ্বার; সব অনুরোধ এখান দিয়ে যায়।	kubectl সবসময় API server-এর সাথে কথা বলে, সরাসরি etcd-র সাথে নয়।
5	**ApplicationSet**	Argo CD-তে টেমপ্লেট থেকে অনেক Application তৈরি করার নিয়ন্ত্রক।	TechNova একই অ্যাপ ৫টি ক্লাস্টারে ApplicationSet দিয়ে চালায়।
6	**Argo CD**	GitOps-ভিত্তিক continuous delivery টুল; Git-এর অবস্থা ক্লাস্টারে প্রয়োগ করে।	technova.example-র প্রোডাকশন ডেপ্লয় Argo CD-র automated sync দিয়ে চলে।
7	**Argo Rollouts**	Progressive delivery নিয়ন্ত্রক; canary ও blue-green rollout পরিচালনা করে।	নতুন রিলিজ 10% ট্রাফিক দিয়ে শুরু করতে Argo Rollouts-এর analysis ব্যবহার করা হয়।
8	**Backstage**	Spotify-র তৈরি developer portal কাঠামো; সার্ভিস ক্যাটালগ ও টেমপ্লেট দেয়।	TechNova-র ডেভেলপাররা Backstage থেকে নতুন সার্ভিস তৈরি করেন।
9	**Blast Radius**	একটি ব্যর্থতা বা পরিবর্তন কতদূর ছড়াতে পারে, তার পরিধি।	Multi-cluster নকশায় প্রতিটি ক্লাস্টারের blast radius আলাদা রাখা হয়।
10	**Blue-Green Deployment**	নতুন সংস্করণ পুরো পরিবেশে বানিয়ে একসাথে ট্রাফিক সুইচ করার কৌশল।	গুরুত্বপূর্ণ রিলিজে TechNova blue-green ব্যবহার করে, যাতে ফিরে যাওয়া দ্রুত হয়।

11	**Burn Rate**	Error budget কত দ্রুত শেষ হচ্ছে, তার হার।	Burn rate 10x হলে বাজেট কয়েক ঘণ্টায় শেষ হয়ে যাবে — দ্রুত alert দরকার।
12	**Canary Deployment**	নতুন সংস্করণে অল্প ট্রাফিক পাঠিয়ে যাচাই করার কৌশল।	5% canary-তে ত্রুটি না বাড়লে ট্রাফিক ধাপে ধাপে 100% করা হয়।
13	**Cardinality**	একটি মেট্রিকের সম্ভাব্য লেবেল মানের সংখ্যা।	User ID লেবেলে রাখলে cardinality বিস্ফোরিত হয়ে খরচ বাড়ে।
14	**Chaos Engineering**	নিয়ন্ত্রিতভাবে ব্যর্থতা ঘটিয়ে সিস্টেমের দুর্বলতা খোঁজার চর্চা।	GameDay-তে একটি নোড বন্ধ করে দেখা হয় সেবা টিকে থাকে কি না।
15	**Chargeback**	খরচ আসলেই দলের বাজেট থেকে কাটা।	FinOps-এ chargeback দলকে খরচের দায়িত্ব নিতে বাধ্য করে।
16	**Cluster Autoscaler**	Pod বসানোর জায়গা না থাকলে নোড বাড়ানো বা কমানোর উপাদান।	সকালে ট্রাফিক বাড়লে Cluster Autoscaler নতুন নোড যোগ করে।
17	**CNI (Container Network Interface)**	কনটেইনারের নেটওয়ার্কিংয়ের মান; প্লাগইন দিয়ে Pod নেটওয়ার্ক দেওয়া হয়।	Calico ও Cilium দুটি জনপ্রিয় CNI প্লাগইন।
18	**ConfigMap**	Pod-এ কনফিগারেশন (non-secret) দেওয়ার Kubernetes রিসোর্স।	অ্যাপের পরিবেশ-নির্ভর সেটিংস ConfigMap-এ রাখা হয়।
19	**Control Plane**	ক্লাস্টার পরিচালনা করা উপাদানগুলো — API server, scheduler, etcd।	Control plane অচল হলেও চলমান Pod কিছুক্ষণ চলতে থাকে।
20	**Controller**	কাঙ্ক্ষিত অবস্থা ও বাস্তব অবস্থার মিল রাখতে ক্রমাগত কাজ করা লুপ।	Deployment controller replica সংখ্যা বজায় রাখে।
21	**CRD (Custom Resource Definition)**	Kubernetes-এ নিজের রিসোর্স ধরন যুক্ত করার উপায়।	Argo CD-র Application একটি CRD।
22	**CSI (Container Storage Interface)**	স্টোরেজ সিস্টেমকে Kubernetes-এর সাথে যুক্ত করার মান।	ক্লাউডের ব্লক স্টোরেজ CSI driver দিয়ে PVC-তে যুক্ত হয়।
23	**DaemonSet**	প্রতিটি (বা নির্বাচিত) নোডে একটি করে Pod চালানোর নিয়ন্ত্রক।	লগ সংগ্রহের agent DaemonSet দিয়ে প্রতিটি নোডে চলে।
24	**Data Plane**	ট্রাফিক বহন করা স্তর — Pod, proxy, নেটওয়ার্ক।	Service mesh-এ sidecar proxy data plane-এ থাকে।
25	**Deployment**	Stateless অ্যাপের replica ও rollout পরিচালনা করা নিয়ন্ত্রক।	ওয়েব অ্যাপের নতুন সংস্করণ Deployment-এর rolling update দিয়ে যায়।
26	**Desired State**	Git বা YAML-এ ঘোষিত কাঙ্ক্ষিত অবস্থা।	GitOps-এ Git-ই desired state-এর উৎস।
27	**Disaster Recovery (DR)**	বড় বিপর্যয়ের পর সেবা ফিরিয়ে আনার পরিকল্পনা ও প্রক্রিয়া।	DR মহড়ায় একটি ক্লাস্টার বন্ধ করে failover পরীক্ষা করা হয়।
28	**Drift**	বাস্তব অবস্থা কাঙ্ক্ষিত অবস্থা থেকে সরে যাওয়া।	কেউ হাতে ফায়ারওয়াল বদলালে drift তৈরি হয়; GitOps তা ধরে।

29	**etcd**	Kubernetes-এর distributed key-value store; ক্লাস্টারের অবস্থা রাখে।	etcd-র snapshot থেকে ক্লাস্টারের অবস্থা পুনরুদ্ধার করা যায়।
30	**Error Budget**	SLO-র অনুমোদিত ত্রুটির পরিমাণ — কতটুকু ব্যর্থতা সহ্য করা যায়।	30 দিনে 99.99% SLO-র error budget 4 মিনিট 19 সেকেন্ড।
31	**External Secrets Operator**	বাইরের ভল্ট থেকে সিক্রেট Kubernetes Secret-এ সিঙ্ক করা টুল।	TechNova HashiCorp Vault থেকে সিক্রেট External Secrets দিয়ে আনে।
32	**Fail-Closed**	নিয়ন্ত্রণ ব্যর্থ হলে অনুরোধ প্রত্যাখ্যান করা।	নিরাপত্তা নীতিতে fail-closed নিরাপদ।
33	**Fail-Open**	নিয়ন্ত্রণ ব্যর্থ হলে অনুরোধ গ্রহণ করা।	কম ঝুঁকির নীতিতে কখনো fail-open বিবেচনা করা হয়।
34	**Failover**	প্রাথমিক সিস্টেম ব্যর্থ হলে বিকল্প সিস্টেমে সরে যাওয়া।	DNS failover-এ ট্রাফিক দ্বিতীয় ক্লাস্টারে যায়।
35	**Flagger**	Progressive delivery টুল; canary rollout স্বয়ংক্রিয় করে।	Flagger মেট্রিক দেখে canary এগোয় নাকি ফিরে যায়, তা ঠিক করে।
36	**Flux CD**	GitOps Toolkit-ভিত্তিক continuous delivery টুল।	Flux প্রতি কয়েক মিনিটে Git-এর সাথে ক্লাস্টারের মিল যাচাই করে।
37	**GameDay**	নিয়ন্ত্রিত বিপর্যয় মহড়া; দল একসাথে ব্যর্থতা সামলানোর অনুশীলন করে।	প্রতি ত্রৈমাসিকে TechNova একটি GameDay করে।
38	**Gateway API**	Kubernetes-এ Ingress-এর পরবর্তী মান; ভূমিকা-ভিত্তিক রাউটিং দেয়।	Gateway API-তে HTTPRoute দিয়ে ট্রাফিক নিয়ম লেখা হয়।
39	**GitOps**	Git-কে কাঙ্ক্ষিত অবস্থার উৎস ধরে অবকাঠামো ও অ্যাপ পরিচালনার চর্চা।	TechNova-র সব ডেপ্লয় GitOps দিয়ে হয়; হাতে kubectl নিষিদ্ধ।
40	**Golden Path**	দলের অনুমোদিত, পরীক্ষিত প্রমিত পথ — নতুন সার্ভিস থেকে ডেপ্লয় পর্যন্ত।	Backstage টেমপ্লেট golden path বাস্তবায়ন করে।
41	**Helm**	Kubernetes প্যাকেজ ব্যবস্থাপক; chart দিয়ে অ্যাপ বান্ডল করা হয়।	তৃতীয় পক্ষের অ্যাপ Helm chart দিয়ে ইনস্টল করা হয়।
42	**Helm Chart**	Helm-এর প্যাকেজ — টেমপ্লেট, মান ও মেটাডেটার সংগ্রহ।	Chart-এর values.yaml দিয়ে পরিবেশভেদে মান বদলানো যায়।
43	**HPA (Horizontal Pod Autoscaler)**	মেট্রিক দেখে Pod replica বাড়ানো-কমানো।	CPU 70% ছাড়ালে HPA replica বাড়ায়।
44	**IDP (Internal Developer Platform)**	ডেভেলপারদের স্ব-সেবা (self-service) প্ল্যাটফর্ম।	Backstage TechNova-র IDP-র মুখ।
45	**Idempotence**	একই কাজ যতবারই করা হোক, প্রথমবারের পর আর পরিবর্তন হয় না।	Terraform apply দ্বিতীয়বার চালালে কিছু বদলায় না।
46	**Image Policy**	কোন image সংস্করণ ব্যবহার করা যাবে, তা নির্ধারণ করা নিয়ম।	Flux Image Automation image policy দিয়ে নতুন tag বেছে নেয়।

47	**Immutability**	চলমান জিনিস বদলানোর বদলে নতুন করে বানানো।	কনটেইনার image বদলানো হয় না; নতুন image দিয়ে Pod বানানো হয়।
48	**Ingress**	ক্লাস্টারের বাইরে থেকে HTTP/HTTPS ট্রাফিক রাউট করার রিসোর্স।	technova.example-র ট্রাফিক Ingress দিয়ে ওয়েব সার্ভিসে যায়।
49	**Istio**	জনপ্রিয় service mesh; ট্রাফিক, নিরাপত্তা ও পর্যবেক্ষণ দেয়।	Istio-র control plane-এর নাম Istiod।
50	**Kustomize**	YAML প্যাচ-ভিত্তিক কাস্টমাইজেশন টুল; মূল ফাইল না বদলে overlay দেয়।	Staging overlay দিয়ে replica সংখ্যা বদলানো হয়।
51	**Kyverno**	Kubernetes নেটিভ নীতি ইঞ্জিন; YAML-এ নীতি লেখা যায়।	Kyverno দিয়ে runAsNonRoot বাধ্যতামূলক করা হয়েছে।
52	**Label**	রিসোর্সে যুক্ত key-value চিহ্ন; নির্বাচন ও বরাদ্দে ব্যবহৃত হয়।	app=web লেবেল দিয়ে Service Pod খুঁজে পায়।
53	**LimitRange**	Namespace-এ resource-এর সীমা নির্ধারণ করা।	LimitRange ছাড়া একটি Pod সব resource নিয়ে নিতে পারে।
54	**Liveness Probe**	কনটেইনার বেঁচে আছে কি না, তা পরীক্ষা করা।	Liveness ব্যর্থ হলে kubelet কনটেইনার পুনরায় চালু করে।
55	**Load Balancer**	ট্রাফিক একাধিক লক্ষ্যে ভাগ করা।	ক্লাউড Load Balancer API server-এর সামনে বসে।
56	**mTLS (Mutual TLS)**	দুই পক্ষই একে অপরের পরিচয় যাচাই করা TLS।	Service mesh-এ mTLS দিয়ে সার্ভিস-টু-সার্ভিস যোগাযোগ নিরাপদ হয়।
57	**Multi-Cluster**	একাধিক Kubernetes ক্লাস্টার একসাথে পরিচালনা করা।	TechNova ঢাকা ও চট্টগ্রামে দুটি ক্লাস্টার চালায়।
58	**Mutating Webhook**	অনুরোধ সংরক্ষণের আগে তা বদলে দেওয়া admission webhook।	Sidecar ইনজেকশন একটি mutating webhook দিয়ে হয়।
59	**Namespace**	একটি ক্লাস্টারের ভেতরে যৌক্তিক বিভাজন।	প্রতিটি দলের জন্য আলাদা Namespace রাখা হয়।
60	**NetworkPolicy**	Pod-এর মধ্যে নেটওয়ার্ক ট্রাফিকের অনুমতি নিয়ন্ত্রণ করা।	NetworkPolicy দিয়ে ডেটাবেস শুধু অ্যাপ Pod থেকে পৌঁছানো যায়।

হ্যান্ডবুক • শব্দকোষ ৬১–১১০

Glossary — Advanced DevOps শব্দকোষ (৬১–১১০)

শব্দ 61–110। প্রতিটি শব্দে ইংরেজি পরিভাষা, বাংলা ব্যাখ্যা ও বাস্তব প্রসঙ্গ দেওয়া আছে।

ক্রম	পরিভাষা (English)	বাংলা ব্যাখ্যা	বাস্তব প্রসঙ্গ / উদাহরণ
61	**Node**	ক্লাস্টারের একটি কর্মী মেশিন; Pod এখানে চলে।	নোডে জায়গা না থাকলে Pod Pending হয়।
62	**Observability**	সিস্টেমের ভেতরের অবস্থা বাইরে থেকে বোঝার ক্ষমতা।	লগ, মেট্রিক ও ট্রেস একসাথে observability দেয়।
63	**OPA (Open Policy Agent)**	সাধারণ নীতি ইঞ্জিন; Rego ভাষায় নীতি লেখা হয়।	OPA দিয়ে Terraform পরিকল্পনাও যাচাই করা যায়।
64	**OpenTelemetry**	Vendor-neutral observability ডেটা সংগ্রহের মান ও SDK।	একটি SDK দিয়ে trace যেকোনো backend-এ পাঠানো যায়।
65	**Overlay**	Kustomize-এ পরিবেশভেদে পরিবর্তনের স্তর।	Prod overlay-এ replica 3, dev overlay-এ 1।
66	**PeerAuthentication**	Istio-তে mTLS মোড নির্ধারণ করা রিসোর্স।	STRICT মোডে sidecar ছাড়া যোগাযোগ ব্যর্থ হয়।
67	**PersistentVolume (PV)**	ক্লাস্টারে স্টোরেজের বাস্তব অংশ।	PV-র জীবন Pod-এর চেয়ে দীর্ঘ।
68	**PersistentVolumeClaim (PVC)**	স্টোরেজ চাওয়ার অনুরোধ।	ডেটাবেস Pod PVC দিয়ে স্টোরেজ চায়।
69	**Platform Engineering**	ডেভেলপারদের জন্য স্ব-সেবা প্ল্যাটফর্ম বানানোর শৃঙ্খলা।	প্ল্যাটফর্ম দল golden path ও টেমপ্লেট বানায়।
70	**Pod**	Kubernetes-এ সবচেয়ে ছোট স্থাপন-যোগ্য একক; এক বা একাধিক কনটেইনার।	Pod-ই scheduling-এর একক।
71	**PodDisruptionBudget (PDB)**	স্বেচ্ছায় ব্যাহত করার সময় ন্যূনতম কত Pod চালু থাকতে হবে, তা নির্ধারণ করে।	আপগ্রেডে PDB অন্তত 1টি Pod চালু রাখে।
72	**Policy as Code**	নীতি কোড হিসেবে লেখা, সংস্করণ নিয়ন্ত্রণে রাখা ও স্বয়ংক্রিয়ভাবে প্রয়োগ করা।	OPA নীতি Git-এ থাকে, PR-এ পর্যালোচনা হয়।
73	**Progressive Delivery**	ধাপে ধাপে, যাচাই করে নতুন সংস্করণ ছাড়ার কৌশল।	Canary ও blue-green দুটোই progressive delivery।
74	**Quorum**	সিদ্ধান্ত নিতে প্রয়োজনীয় ন্যূনতম সদস্য সংখ্যা।	তিন সদস্যের etcd-তে quorum 2।

75	**RBAC (Role-Based Access Control)**	ভূমিকা অনুযায়ী অনুমতি দেওয়ার ব্যবস্থা।	ডেভেলপার শুধু নিজের Namespace-এ Pod দেখতে পারেন।
76	**Readiness Probe**	কনটেইনার ট্রাফিক নেওয়ার জন্য প্রস্তুত কি না, তা পরীক্ষা করা।	Readiness ব্যর্থ হলে Service Pod-এ ট্রাফিক পাঠায় না।
77	**Reconciliation**	কাঙ্ক্ষিত অবস্থার সাথে বাস্তব অবস্থা মেলানোর ক্রমাগত প্রক্রিয়া।	Flux প্রতি কয়েক মিনিটে reconciliation চালায়।
78	**ResourceQuota**	Namespace-এ মোট resource ব্যবহারের সীমা।	Quota ছাড়া একটি দল পুরো ক্লাস্টারের resource নিয়ে নিতে পারে।
79	**Rollout**	নতুন সংস্করণ ধাপে ধাপে চালু করার প্রক্রিয়া।	Rolling update-এ Pod একে একে বদলানো হয়।
80	**Rollback**	আগের পরিচিত ভালো সংস্করণে ফিরে যাওয়া।	Canary ব্যর্থ হলে স্বয়ংক্রিয় rollback হয়।
81	**RPO (Recovery Point Objective)**	কতটুকু ডেটা হারানো সহ্য করা যায়।	RPO 1 ঘণ্টা মানে সর্বোচ্চ 1 ঘণ্টার ডেটা হারাতে পারে।
82	**RTO (Recovery Time Objective)**	বিপর্যয়ের পর কতক্ষণে সেবা ফিরতে হবে।	RTO 4 ঘণ্টা মানে 4 ঘণ্টার মধ্যে সেবা ফিরতে হবে।
83	**SBOM (Software Bill of Materials)**	সফটওয়্যারের উপাদান ও নির্ভরতার তালিকা।	CVE প্রকাশ পেলে SBOM দিয়ে প্রভাবিত সার্ভিস খুঁজে পাওয়া যায়।
84	**Scheduler**	Pod-কে কোন নোডে বসাবে, তা ঠিক করা উপাদান।	Scheduler resource request, affinity ও taint দেখে সিদ্ধান্ত নেয়।
85	**Sealed Secrets**	এনক্রিপ্টেড সিক্রেট Git-এ নিরাপদে রাখার উপায়।	SealedSecret শুধু ক্লাস্টারের ভেতরে ডিক্রিপ্ট হয়।
86	**Secret**	সংবেদনশীল তথ্য (পাসওয়ার্ড, টোকেন) রাখার Kubernetes রিসোর্স।	Secret base64-এ থাকে — এনক্রিপশন নয়, তাই Git-এ রাখা যায় না।
87	**Service**	Pod-এর সামনে স্থিতিশীল নেটওয়ার্ক পরিচয় দেওয়া রিসোর্স।	Pod বদলালেও Service-র নাম ও IP স্থির থাকে।
88	**Service Mesh**	সার্ভিস-টু-সার্ভিস যোগাযোগ পরিচালনা করা অবকাঠামো স্তর।	Istio ও Linkerd দুটি জনপ্রিয় service mesh।
89	**Showback**	খরচ দলকে দেখানো, কিন্তু টাকা না কাটা।	Showback দিয়ে শুরু করে খরচের দায়িত্বের সংস্কৃতি গড়া যায়।
90	**Sidecar**	মূল কনটেইনারের পাশে চলা সহায়ক কনটেইনার।	Service mesh-এ proxy sidecar হিসেবে চলে।
91	**SLA (Service Level Agreement)**	গ্রাহকের সাথে চুক্তিগত সেবার প্রতিশ্রুতি।	SLA ভাঙলে জরিমানা হতে পারে।
92			SLO ব্যবসায়িক প্রয়োজন দিয়ে ঠিক হয়।

	SLO (Service Level Objective)	সেবার নির্ভরযোগ্যতার লক্ষ্য, যেমন 99.9% উপলব্ধতা।	
93	**SLI (Service Level Indicator)**	সেবার নির্ভরযোগ্যতার পরিমাপ।	সফল অনুরোধের হার একটি SLI।
94	**Span**	একটি trace-এর একটি অংশ; একটি সার্ভিসে একটি কাজ।	Span-গুলো মিলে অনুরোধের পুরো পথ দেখায়।
95	**Split-Brain**	দুটি অংশ একে অপরকে না দেখে আলাদা সিদ্ধান্ত নেওয়া।	etcd quorum হারালে লেখা বন্ধ করে split-brain এড়ায়।
96	**StatefulSet**	স্থিতিশীল পরিচয় ও স্টোরেজসহ Pod পরিচালনা করা নিয়ন্ত্রক।	PostgreSQL StatefulSet দিয়ে চলে।
97	**StorageClass**	স্টোরেজের ধরন ও provisioner নির্ধারণ করা।	দ্রুত SSD ও সাধারণ ডিস্কের জন্য আলাদা StorageClass থাকে।
98	**Taint & Toleration**	নোডে Pod বসা সীমিত করার ব্যবস্থা।	GPU নোডে taint থাকে; শুধু toleration-সহ Pod বসতে পারে।
99	**Terraform**	HashiCorp-এর IaC টুল; ঘোষণামূলক ভাষায় অবকাঠামো বানানো হয়।	TechNova-র নেটওয়ার্ক ও ক্লাস্টার Terraform দিয়ে বানানো।
100	**Trace**	একটি অনুরোধের পুরো পথের রেকর্ড; অনেক span-এর সংগ্রহ।	ধীর অনুরোধের কারণ trace দেখে খুঁজে পাওয়া যায়।
101	**Validating Webhook**	অনুরোধ নীতি মেনে চলে কি না, তা যাচাই করা admission webhook।	নীতি-বিরুদ্ধ Pod validating webhook আটকে দেয়।
102	**Vendor Lock-in**	একটি বিক্রেতার উপর এমন নির্ভরতা, যাতে সরে যাওয়া কঠিন হয়।	খোলা মান ব্যবহার করলে lock-in কমে।
103	**VirtualService**	Istio-তে ট্রাফিক রাউটিং ও ভাগ করার নিয়ম।	Canary-তে VirtualService-এ weight দিয়ে ট্রাফিক ভাগ করা হয়।
104	**VPA (Vertical Pod Autoscaler)**	Pod-এর CPU/memory request ও limit বদলে দেওয়া।	VPA সুপারিশ প্রয়োগ করতে Pod পুনরায় চালু করে।
105	**VolumeClaimTemplate**	StatefulSet-এ প্রতিটি Pod-এর জন্য PVC তৈরির টেমপ্লেট।	প্রতিটি ডেটাবেস replica নিজের PVC পায়।
106	**Workload Identity**	Pod-কে ক্লাউড পরিচয় দেওয়া, দীর্ঘমেয়াদি চাবি ছাড়া।	Workload identity দিয়ে Pod নিরাপদে ক্লাউড সেবা ডাকে।
107	**Zero-Trust**	কাউকে অবস্থান দিয়ে বিশ্বাস নয়; প্রতিটি অনুরোধ যাচাই করা।	Zero-trust-এ ভেতরের নেটওয়ার্কও স্বয়ংক্রিয়ভাবে নিরাপদ নয়।
108	**Admission Review**	Admission webhook-এ পাঠানো অনুরোধের তথ্য।	Webhook AdmissionReview দেখে সিদ্ধান্ত নেয়।
109	**Aggregation**		

		ছোট ছোট অংশ একত্র করে বড় চিত্র তৈরি করা।	মেট্রিক aggregation-এ অনেক Pod-এর ডেটা একসাথে দেখা যায়।
110	**Alert Fatigue**	অতিরিক্ত alert-এ দল সতর্কতা উপেক্ষা করতে শুরু করা।	Burn rate alert দিয়ে শুধু গুরুত্বপূর্ণ সতর্কতা পাঠানো হয়।

হ্যান্ডবুক • শব্দকোষ ১১১–১৫০

Glossary — Advanced DevOps শব্দকোষ (১১১–১৫০)

শব্দ 111–150। প্রতিটি শব্দে ইংরেজি পরিভাষা, বাংলা ব্যাখ্যা ও বাস্তব প্রসঙ্গ দেওয়া আছে।

ক্রম	পরিভাষা (English)	বাংলা ব্যাখ্যা	বাস্তব প্রসঙ্গ / উদাহরণ
111	**Audit Log**	কে, কখন, কী করল — তার রেকর্ড।	API server-এর audit log নিরাপত্তা তদন্তে ব্যবহৃত হয়।
112	**Availability Zone**	একটি অঞ্চলের ভেতরে আলাদা ব্যর্থতা-ক্ষেত্র।	তিনটি zone-এ replica ছড়িয়ে রাখলে একটি zone ব্যর্থ হলেও সেবা চলে।
113	**Backoff**	ব্যর্থতার পর আবার চেষ্টার আগে অপেক্ষা বাড়ানো।	CrashLoopBackOff-এ পুনরায় চালুর ব্যবধান ধীরে ধীরে বাড়ে।
114	**Capacity Planning**	ভবিষ্যতের চাহিদা অনুযায়ী resource পরিকল্পনা করা।	Black Friday-এর আগে capacity planning করা হয়।
115	**CI/CD**	Continuous Integration ও Continuous Delivery/Deployment।	PR মার্জ হলেই CI পরীক্ষা চালায়, CD ডেপ্লয় করে।
116	**Config Drift**	কনফিগারেশন সময়ের সাথে কাঙ্ক্ষিত অবস্থা থেকে সরে যাওয়া।	GitOps config drift ধরে ও সংশোধন করে।
117	**Container Runtime**	কনটেইনার চালানোর সফটওয়্যার, যেমন containerd।	kubelet container runtime-কে Pod চালাতে বলে।
118	**CrashLoopBackOff**	কনটেইনার বারবার চালু হয়ে ব্যর্থ হওয়া।	ভুল image বা কনফিগারেশনে CrashLoopBackOff হয়।
119	**Custom Resource**	CRD দিয়ে তৈরি নিজের ধরনের রিসোর্স।	Application একটি custom resource।
120	**Daemon**	পেছনে চলা সেবা প্রক্রিয়া।	প্রতিটি নোডে kubelet একটি daemon হিসেবে চলে।
121	**Dependency**	একটি উপাদান অন্য উপাদানের উপর নির্ভর করা।	Helm chart-এর dependency অন্য chart হতে পারে।
122	**Deployment Strategy**	নতুন সংস্করণ চালু করার পদ্ধতি — rolling, blue-green, canary।	কৌশল বেছে নেওয়া ঝুঁকি ও খরচের হিসাব।
123	**Endpoint**	একটি সেবা পৌঁছানোর ঠিকানা।	Service-র endpoint হলো তার Pod-গুলোর ঠিকানা।
124	**Ephemeral Volume**	Pod-এর জীবনের সাথে শেষ হয়ে যাওয়া অস্থায়ী স্টোরেজ।	স্থায়ী ডেটা ephemeral volume-এ রাখা যায় না।
125	**Federation**	একাধিক ক্লাস্টারকে একসাথে পরিচালনা করার ব্যবস্থা।	Federation জটিল; অনেক দল স্বাধীন ক্লাস্টার বেছে নেয়।

126	**Git Repository**	কোড ও কনফিগারেশনের সংস্করণ-নিয়ন্ত্রিত সংগ্রহ।	GitOps-এ Git repository-ই সত্যের উৎস।
127	**Health Check**	সেবা সুস্থ আছে কি না, তা পরীক্ষা করা।	Load balancer health check ব্যর্থ হলে ট্রাফিক সরিয়ে নেয়।
128	**High Availability (HA)**	ব্যর্থতা সত্ত্বেও সেবা চালু থাকা।	তিনটি etcd সদস্য HA দেয়।
129	**Horizontal Scaling**	সংখ্যা বাড়িয়ে স্কেল করা — বেশি Pod বা নোড।	HPA horizontal scaling করে।
130	**Immutable Infrastructure**	বদলানোর বদলে নতুন করে বানানো অবকাঠামো।	নতুন image দিয়ে নতুন Pod — পুরোনোটি বদলানো হয় না।
131	**Infrastructure as Code (IaC)**	অবকাঠামো কোড হিসেবে লেখা ও পরিচালনা করা।	Terraform দিয়ে IaC চর্চা করা হয়।
132	**Kubelet**	প্রতিটি নোডে চলা agent; Pod চালানো ও পর্যবেক্ষণ করা।	kubelet API server-এর নির্দেশে Pod চালায়।
133	**Kube-Proxy**	প্রতিটি নোডে Service-র নেটওয়ার্ক নিয়ম বাস্তবায়ন করা।	kube-proxy Service-র ট্রাফিক Pod-এ পাঠায়।
134	**Latency**	অনুরোধের উত্তরে কত সময় লাগে।	P99 latency মানে 99% অনুরোধ এই সময়ের মধ্যে শেষ হয়।
135	**Leader Election**	একাধিক replica-র মধ্যে একজন নেতা বেছে নেওয়া।	etcd-তে leader election দিয়ে একজন নেতা লেখা পরিচালনা করে।
136	**Load Testing**	চাপ দিয়ে সিস্টেমের সীমা পরীক্ষা করা।	Black Friday-এর আগে load testing করা হয়।
137	**Maturity Model**	একটি দল বা প্ল্যাটফর্ম কতটা পরিণত, তা মাপার কাঠামো।	Platform maturity assessment দিয়ে পরের ধাপ ঠিক করা হয়।
138	**Metrics Server**	Kubernetes-এ resource মেট্রিক দেওয়া উপাদান।	HPA Metrics Server থেকে CPU মেট্রিক পায়।
139	**Multi-Tenancy**	একটি ক্লাস্টারে একাধিক দলের নিরাপদ সহাবস্থান।	Namespace, quota ও নীতি দিয়ে multi-tenancy নিশ্চিত করা হয়।
140	**Node Pool**	একই ধরনের নোডের গোষ্ঠী।	GPU কাজের জন্য আলাদা node pool রাখা হয়।
141	**Operator**	Kubernetes-এ জটিল অ্যাপ পরিচালনা করা নিয়ন্ত্রক।	PostgreSQL operator ব্যাকআপ ও failover সামলায়।
142	**Orphan Resource**	Git-এ নেই, কিন্তু ক্লাস্টারে আছে এমন রিসোর্স।	Argo CD prune orphan resource মুছে দেয়।
143	**Persistent Storage**	Pod মুছলেও টিকে থাকা স্টোরেজ।	ডেটাবেসের ডেটা persistent storage-এ থাকে।

144	**Pod Template**	Pod তৈরির ছক; Deployment ও StatefulSet এটি ব্যবহার করে।	টেমপ্লেট বদলালে নতুন Pod তৈরি হয়।
145	**Preemption**	উচ্চ অগ্রাধিকারের Pod-এর জন্য নিম্ন অগ্রাধিকারের Pod সরানো।	Spot instance-এ preemption হতে পারে।
146	**PriorityClass**	Pod-এর অগ্রাধিকার নির্ধারণ করা।	গুরুত্বপূর্ণ সেবার PriorityClass বেশি রাখা হয়।
147	**Prune**	Git-এ নেই এমন রিসোর্স ক্লাস্টার থেকে মুছে দেওয়া।	Argo CD-র prune sync-এ orphan resource মুছে যায়।
148	**Readiness Gate**	Pod প্রস্তুত কি না, তা নির্ধারণের অতিরিক্ত শর্ত।	Service mesh-এ readiness gate proxy প্রস্তুত কি না, তা দেখে।
149	**Reconciliation Loop**	কাঙ্ক্ষিত ও বাস্তব অবস্থা মেলানোর ক্রমাগত চক্র।	সব Kubernetes controller reconciliation loop চালায়।
150	**Replica**	একটি Pod বা সার্ভিসের একটি কপি।	Replica 3 মানে একই Pod তিনটি চলছে।

হ্যান্ডবুক • দ্রুত নির্দেশিকা

Quick Reference — Kubernetes Architecture + GitOps Commands

Kubernetes — দ্রুত পরীক্ষা

```
kubectl get nodes -o wide
kubectl get pods -A --field-selector status.phase!=Running
kubectl describe pod <pod> -n <ns>
kubectl logs -f deployment/web -n prod --tail=100
kubectl top nodes && kubectl top pods -A
kubectl get events -A --sort-by=.lastTimestamp | tail -30
kubectl auth can-i create deployments -n prod --as=dev-user
etcdctl endpoint health --cluster
etcdctl snapshot save /backup/etcd-$(date +%F).db
```

> **WARNING:** etcd snapshot নেওয়ার আগে endpoint health যাচাই করুন; দূষিত snapshot থেকে পুনরুদ্ধার করা যায় না। Snapshot নিরাপদ জায়গায় রাখুন ও পুনরুদ্ধার মহড়া করুন।

Argo CD — দ্রুত নির্দেশিকা

```
argocd app list
argocd app get web-prod
argocd app sync web-prod --prune
argocd app diff web-prod
argocd app history web-prod
argocd app rollback web-prod 3
argocd admin dashboard
kubectl -n argocd get applications
```

Flux — দ্রুত নির্দেশিকা

```
flux check
flux get sources git -A
flux get kustomizations -A
flux reconcile source git technova
flux reconcile kustomization apps --with-source
flux suspend kustomization apps
```

```
flux resume kustomization apps
flux logs --level=error -A
```

> **CONCEPT:** Suspend করা মানে reconciliation থামানো, মোছা নয়। জরুরি সময়ে suspend করে হাতে কাজ করুন, তারপর resume করুন — নইলে Flux আপনার হাতের পরিবর্তন ফিরিয়ে দেবে।

হ্যান্ডবুক • তুলনা ও সিদ্ধান্ত

Quick Reference — Argo CD বনাম Flux, Helm বনাম Kustomize, Backstage, Service Mesh, Progressive Delivery

Argo CD বনাম Flux

বিষয়	Argo CD	Flux
মূল ধারণা	Application CRD-কেন্দ্রিক; UI সমৃদ্ধ	GitOps Toolkit; Kubernetes নেটিভ রিসোর্স
UI	শক্তিশালী ওয়েব UI, diff দৃশ্যমান	সীমিত; CLI ও Git-কেন্দ্রিক
Multi-cluster	ApplicationSet দিয়ে টেমপ্লেট	সোর্স ও Kustomization দিয়ে গঠন
শেখা	UI দিয়ে শুরু করা সহজ	Kubernetes ধারণা জানা থাকলে সহজ
কখন বেছে নেবেন	দল UI ও দৃশ্যমানতা চাইলে	দল CLI ও কম্পোজেবল টুলকিট চাইলে

Helm বনাম Kustomize

বিষয়	Helm	Kustomize
পদ্ধতি	টেমপ্লেট + values	মূল YAML + overlay প্যাচ
শক্তি	প্যাকেজ করা অ্যাপ, release ব্যবস্থাপনা, rollback	নিজের YAML-এ পরিবেশভেদে পরিবর্তন
দুর্বলতা	টেমপ্লেট জটিল হতে পারে	Release ইতিহাস ও rollback সীমিত
কখন বেছে নেবেন	তৃতীয় পক্ষের অ্যাপ বা পুনর্ব্যবহারযোগ্য প্যাকেজে	নিজের অ্যাপের পরিবেশভেদে কনফিগারেশনে

দুটো একসাথে চলে: Helm chart-কে Kustomize দিয়ে প্যাচ করা যায়, আবার Argo CD/Flux দুটোই সমর্থন করে।

Backstage Architecture — সংক্ষেপে

Backstage-এর তিনটি মূল অংশ: **Software Catalog** (সব সার্ভিসের তালিকা ও মালিকানা), **Software Templates / Scaffolder** (নতুন সার্ভিস বানানোর ছক), এবং **TechDocs** (কোডের সাথে ডকুমেন্টেশন)। Catalog তথ্য আসে `catalog-info.yaml` থেকে — কোডে লেখা তথ্য স্বয়ংক্রিয়ভাবে হালনাগাদ হয়। প্লাগইন দিয়ে Kubernetes, CI/CD ও পর্যবেক্ষণ যুক্ত করা যায়।

Service Mesh — সিদ্ধান্ত নির্দেশিকা

পরিস্থিতি	সুপারিশ
৩–৫টি সার্ভিস, ছোট দল	Mesh নয়; Ingress + TLS + ভালো পর্যবেক্ষণ দিয়ে শুরু করুন।

অনেক সার্ভিস, mTLS ও নীতি দরকার	Mesh বিবেচনা করুন; প্রথমে একটি Namespace-এ পাইলট করুন।
Canary ও ট্রাফিক নিয়ন্ত্রণ দরকার	Mesh-এর VirtualService অথবা Gateway API + Flagger দিয়ে শুরু করুন।
অপারেশন ক্ষমতা সীমিত	সহজ mesh (যেমন Linkerd) অথবা mesh ছাড়া বিকল্প বেছে নিন।

Progressive Delivery — সংক্ষেপে

ধাপ: ১% → ৫% → ১০% → ২৫% → ৫০% → ১০০%। প্রতিটি ধাপে মেট্রিক যাচাই (ত্রুটির হার, লেটেন্সি, ব্যবসায়িক সূচক)। মেট্রিক খারাপ হলে স্বয়ংক্রিয় rollback। মনে রাখবেন: মেট্রিক সংগ্রহে বিলম্ব থাকলে ধাপের সময় বাড়াতে হয় — নইলে ভাঙা রিলিজও এগিয়ে যায়।

হ্যান্ডবুক • চেকলিস্ট ও প্যাটার্ন

Quick Reference — IDP Design, Multi-Cluster Operations, Terraform/Ansible Patterns, Platform Maturity

Internal Developer Platform — নকশা চেকলিস্ট

- ব্যবহারকারীর আসল সমস্যা চিহ্নিত করা হয়েছে কি? (ডেভেলপারদের সাথে কথা বলে)
- Golden path আছে কি? নতুন সার্ভিস বানানো থেকে ডেপ্লয় পর্যন্ত একটানা পথ কি?
- Catalog-এ প্রতিটি সার্ভিসের মালিক (owner) আছে কি?
- টেমপ্লেট মান অনুযায়ী বানানো হয় কি? (CI, পরীক্ষা, পর্যবেক্ষণসহ)
- ডকুমেন্টেশন কোডের সাথে আছে কি? (TechDocs/docs-as-code)
- সাফল্যের সূচক ঠিক করা আছে কি? (সময় বাঁচা, ব্যবহারকারীর সংখ্যা)
- ফিডব্যাকের পথ আছে কি? (ব্যবহারকারীরা সমস্যা জানাতে পারে কি?)

Multi-Cluster Operations — চেকলিস্ট

- প্রতিটি ক্লাস্টারের সংস্করণ ও নীতি একই কি? (GitOps দিয়ে যাচাই)
- Failover পরীক্ষা করা হয়েছে কি? (DNS/Global LB স্বাস্থ্য-পরীক্ষাসহ)
- কেন্দ্রীয় পর্যবেক্ষণ আছে কি? (সব ক্লাস্টারের মেট্রিক এক জায়গায়)
- etcd ব্যাকআপ প্রতিটি ক্লাস্টারে নিয়মিত নেওয়া হয় কি?
- নতুন ক্লাস্টার যোগ করার runbook আছে কি?

Terraform ও Ansible — প্যাটার্ন

```
# Terraform: পরিবেশভেদে শুধু চলক বদলান, মূল কোড নয়
# environments/prod/main.tf  ->  module "network" { source = "../../modules/network" }
# Remote backend + lock চালু রাখুন; state Git-এ তুলবেন না।
# Ansible: idempotent task লিখুন
# - name: Ensure nginx is installed
#   apt: { name: nginx, state: present }
# Vault দিয়ে সিক্রেট এনক্রিপ্ট করুন; inventory গোষ্ঠীভিত্তিক রাখুন।
```

Platform Maturity Assessment

স্তর	বৈশিষ্ট্য	পরের ধাপ
১ — প্রাথমিক	হাতে কাজ, কোনো টেমপ্লেট নেই	একটি টেমপ্লেট ও catalog শুরু করুন

২ — পুনরাবৃত্তিযোগ্য	কিছু টেমপ্লেট আছে, ব্যবহার আংশিক	Golden path একটানা করুন, ফিডব্যাক নিন
৩ — সংজ্ঞায়িত	Golden path ব্যবহৃত হচ্ছে, সূচক আছে	পর্যবেক্ষণ ও নীতি যুক্ত করুন
৪ — পরিমাপযোগ্য	সূচক দিয়ে সিদ্ধান্ত নেওয়া হয়	স্বয়ংক্রিয় উন্নতি ও FinOps যুক্ত করুন
৫ — অনুকূলিত	প্ল্যাটফর্ম নিজেই শেখে ও উন্নত হয়	নতুন দল ও ব্যবহার-ক্ষেত্রে ছড়িয়ে দিন

হ্যান্ডবুক • চেকলিস্ট

Checklists — Production Readiness, FinOps, Security Governance

Production Readiness Checklist

- Resource request ও limit নির্ধারণ করা হয়েছে কি?
- Readiness ও liveness probe আছে কি? Startup সময়ের সাথে মিল আছে কি?
- PodDisruptionBudget আছে কি?
- Replica কমপক্ষে ২টি কি? (একটি নোড ব্যর্থ হলেও সেবা চলে কি?)
- সিক্রেট ভল্ট/External Secrets থেকে আসছে কি? Git-এ প্লেইন টেক্সটে নেই তো?
- NetworkPolicy আছে কি?
- লগ, মেট্রিক ও trace চালু আছে কি?
- Rollback পরিকল্পনা আছে কি? (Git revert বা helm rollback)
- ডেটাবেস ব্যাকআপ ও পুনরুদ্ধার পরীক্ষা করা হয়েছে কি?
- Runbook ও on-call দায়িত্ব লেখা আছে কি?

FinOps Checklist

- প্রতিটি Namespace, দল ও সার্ভিসে ট্যাগ/লেবেল আছে কি?
- খরচের বরাদ্দ (allocation) রিপোর্ট নিয়মিত দেখা হয় কি?
- বাজেট alert বসানো আছে কি?
- Overprovisioned Pod চিহ্নিত করা হয় কি? (ব্যবহার বনাম request)
- অব্যবহৃত resource (যেমন পুরোনো volume, বন্ধ Pod-এর PVC) পরিষ্কার করা হয় কি?
- Spot/preemptible ব্যবহারের নকশা আছে কি?

Security Governance Checklist

- RBAC ন্যূনতম অনুমতি (least privilege) অনুযায়ী আছে কি?
- Admission নীতি (OPA/Kyverno) চালু আছে কি? Audit মোডে পরীক্ষা করা হয়েছে কি?
- Image স্বাক্ষর যাচাই চালু আছে কি?
- সিক্রেট rotation-এর নিয়ম আছে কি?
- Pod Security Standards প্রয়োগ করা হয়েছে কি?
- Audit log সংরক্ষণ ও পর্যালোচনা করা হয় কি?
- নিয়মিত দুর্বলতা (CVE) স্ক্যান করা হয় কি?

REAL WORLD: TechNova-তে প্রোডাকশনে যাওয়ার আগে এই তিনটি চেকলিস্টের প্রতিটি আইটেমে টিক দিতে হয়। একটি আইটেম বাদ গেলেই ডেপ্লয় থামে — কারণ প্রোডাকশনে ধরা পড়া ভুলের খরচ পর্যালোচনায় ধরা পড়া ভুলের চেয়ে অনেক বেশি।

হ্যান্ডবুক • সিদ্ধান্ত গাছ

Troubleshooting Decision Trees (Text/ASCII)

GitOps Sync ব্যর্থতা

```
Argo CD / Flux sync ব্যর্থ?
├── Git-এ পৌঁছানো যাচ্ছে না?
│   ├── হ্যাঁ → Repository URL, credential ও নেটওয়ার্ক যাচাই করুন
│   └── না → পরের ধাপ
├── YAML-এ ত্রুটি আছে?
│   ├── হ্যাঁ → kubectl apply --dry-run দিয়ে যাচাই করুন; লাইন ধরে ঠিক করুন
│   └── না → পরের ধাপ
├── ক্লাস্টারে অনুমতি নেই?
│   ├── হ্যাঁ → ServiceAccount-এর RBAC যাচাই করুন
│   └── না → Argo CD/Flux controller-র লগ দেখুন; status/condition পড়ুন
└── Diff-এ অপ্রত্যাশিত পরিবর্তন?
    └── অ্যাপ নিজে রিসোর্স বদলাচ্ছে কি না, তা যাচাই করুন (ignoreDifferences বিবেচনা)
```

Cluster ব্যর্থতা

```
kubectl সাড়া দিচ্ছে না?
├── API server চলছে কি?
│   ├── না → Control plane নোডের অবস্থা দেখুন; etcd health যাচাই করুন
│   └── হ্যাঁ → পরের ধাপ
├── etcd quorum আছে কি?
│   ├── না → Snapshot থেকে পুনরুদ্ধারের পরিকল্পনা করুন; চলমান Pod চলতে থাকবে
│   └── হ্যাঁ → API server-র লগ ও resource দেখুন
└── শুধু একটি Namespace-এ সমস্যা?
    └── Quota, NetworkPolicy ও admission নীতি যাচাই করুন
```

Progressive Delivery Rollback

```
Canary-তে ত্রুটি বাড়ছে?
├── ত্রুটির হার SLO ছাড়িয়েছে?
│   ├── হ্যাঁ → অবিলম্বে rollback করুন; কারণ পরে খুঁজুন
│   └── না → মেট্রিক বিলম্ব আছে কি না, তা যাচাই করুন
├── মেট্রিক বিলম্ব আছে?
│   ├── হ্যাঁ → ধাপের সময় বাড়ান; দ্রুত promotion বন্ধ করুন
│   └── না → পরের ধাপ
└── ত্রুটি কি নির্দিষ্ট গোষ্ঠীতে সীমাবদ্ধ?
```

├── হ্যাঁ → সেই গোষ্ঠীর ট্রাফিক আলাদা করে বিশ্লেষণ করুন
└── না → `Rollout` থামান (pause) ও বিশ্লেষণ করুন

হ্যান্ডবুক • দক্ষতা নিরীক্ষা

Competency Tracking Sheet — Skills Matrix (Self-Rating)

প্রতিটি দক্ষতায় নিজেকে ১–৫ নম্বর দিন: ১ = শুনেছি, ২ = বুঝি, ৩ = করতে পারি, ৪ = শেখাতে পারি, ৫ = নকশা করতে পারি। প্রতি মাসে আবার মাপুন।

দক্ষতা	অধ্যায়	নিজের নম্বর (১–৫)	প্রমাণ (কী করেছি)	পরের লক্ষ্য
Kubernetes control plane	১–২			
Multi-cluster পরিচালনা	৩			
GitOps (Argo CD / Flux)	৪–৫			
Helm ও Kustomize	৬			
Service mesh	৭			
Networking (CNI, Gateway API)	৮			
Storage (CSI, StatefulSet)	৯			
Autoscaling ও scheduling	১০			
Progressive delivery	১১			
IDP ও Backstage	১২–১৩			
Enterprise CI/CD	১৪			
Terraform ও Ansible	১৫–১৬			
Observability (OpenTelemetry)	১৭			
SLO ও error budget	১৮			
Security governance	১৯			
FinOps	২০			
Disaster recovery ও chaos	২১			
Multi-cloud ও hybrid	২২			
Zero-trust ও secrets	২৩			
Enterprise transformation	২৪			
স্থাপত্য সিদ্ধান্ত রক্ষা করা	সব			

নিয়ম: কোনো দক্ষতায় ৩-এর নিচে থাকলে সংশ্লিষ্ট অধ্যায়ের scenario-গুলো আবার করুন। ৪-এ পৌঁছাতে হলে কাউকে বিষয়টি শেখান — শেখানোই গভীরতার পরীক্ষা।

হ্যান্ডবুক • তথ্যসূত্র

Official References

নিচের সব লিংক অফিসিয়াল ডকুমেন্টেশনের — কোনো বানানো লিংক নেই। সংস্করণ বা দাম এখানে দেওয়া হয়নি; সর্বশেষ তথ্যের জন্য অফিসিয়াল পাতাটিই দেখুন।

- Kubernetes Documentation — https://kubernetes.io/docs/
- Kubernetes Concepts: Control Plane — https://kubernetes.io/docs/concepts/overview/components/
- CNCF — https://www.cncf.io/
- CNCF Landscape — https://landscape.cncf.io/
- Argo CD Documentation — https://argo-cd.readthedocs.io/
- Argo Rollouts Documentation — https://argoproj.github.io/argo-rollouts/
- Flux CD Documentation — https://fluxcd.io/docs/
- Backstage Documentation — https://backstage.io/docs/
- HashiCorp Terraform Documentation — https://developer.hashicorp.com/terraform/docs
- HashiCorp Vault Documentation — https://developer.hashicorp.com/vault/docs
- OpenTelemetry Documentation — https://opentelemetry.io/docs/
- Istio Documentation — https://istio.io/latest/docs/
- Linkerd Documentation — https://linkerd.io/docs/
- Helm Documentation — https://helm.sh/docs/
- Kustomize Documentation — https://kustomize.io/
- Prometheus Documentation — https://prometheus.io/docs/
- Open Policy Agent Documentation — https://www.openpolicyagent.org/docs/
- Kyverno Documentation — https://kyverno.io/docs/
- FinOps Foundation — https://www.finops.org/
- Google SRE Book (SLO/Error Budget) — https://sre.google/sre-book/table-of-contents/
- AWS Architecture Center — https://aws.amazon.com/architecture/
- Microsoft Azure Architecture Center — https://learn.microsoft.com/en-us/azure/architecture/
- Google Cloud Architecture Center — https://cloud.google.com/architecture

> **CONCEPT:** এই ভলিউম এখানেই শেষ নয় — স্থাপত্য শেখা একটি চলমান চর্চা। TechNova-র গল্পে আপনি যা শিখলেন, তা এবার নিজের প্রকল্পে প্রয়োগ করুন: একটি ছোট ক্লাস্টার বানান, GitOps দিয়ে একটি অ্যাপ চালান, একটি canary করুন, একটি GameDay করুন। ভুল করুন, শিখুন, আবার করুন — এটাই Enterprise Platform Engineering।